



FYDP I DEFENSE / UIU • CSE

Solving the Lorenz ODE System using **Optimal ANN Architectures**

Can a plain feedforward ANN, with no physics in the loss, learn a coupled nonlinear ODE system, and which architecture does it best?

Team Paradox

Ikramul Hasan Morad

Md. Abu Bakar

Samiur Rahman Omlan

Fariha Islam

Md. Touhidul Islam

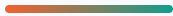
SUPERVISOR Dr. Muhammad Nomani Kabir, Professor, CSE

COURSE TEACHER Dr. Md. Motaharul Islam, Professor, CSE

DATE June 2026

ROADMAP

What we'll cover



01 Problem & Objectives

Why coupled ODEs are hard, and what we set out to do.

02 Background & Gap

The Lorenz system, ANN vs PINN, and the literature gap.

03 Methodology

A two-phase, 69-run controlled architecture search.

04 Numerical Baseline

A dual-solver reference we can trust.

05 Result & Validation

The baseline verified to machine precision.

06 Plan & Conclusion

FYDP-II roadmap and where we stand today.

Three reasons this is hard



No closed form

Coupled nonlinear ODEs like Lorenz-1960 rarely admit an analytical solution.



Solvers are costly

RK4 is accurate but mesh-dependent, and re-solves step by step for every new query.



Neural nets lean on physics

Most neural ODE methods add physics-informed losses or operator learning to get there.

Central question: can a **plain feedforward ANN**, trained only on numerical data with no physics in the loss, approximate the Lorenz-1960 solution, and *which architecture does it best?*

What we set out to do



Build a trusted reference

Solve Lorenz-1960 with RK4 *and* SciPy, then cross-check one solver against the other.



Search the architecture space

Systematically vary **depth**, **width**, and **activation**, with no physics aids in the loss.



Measure accuracy vs cost

MSE, MAE, max error, plus training and inference time and convergence stability.



Benchmark vs literature

Position the best plain ANN against PINN / DeepONet results from prior work.

The Lorenz system

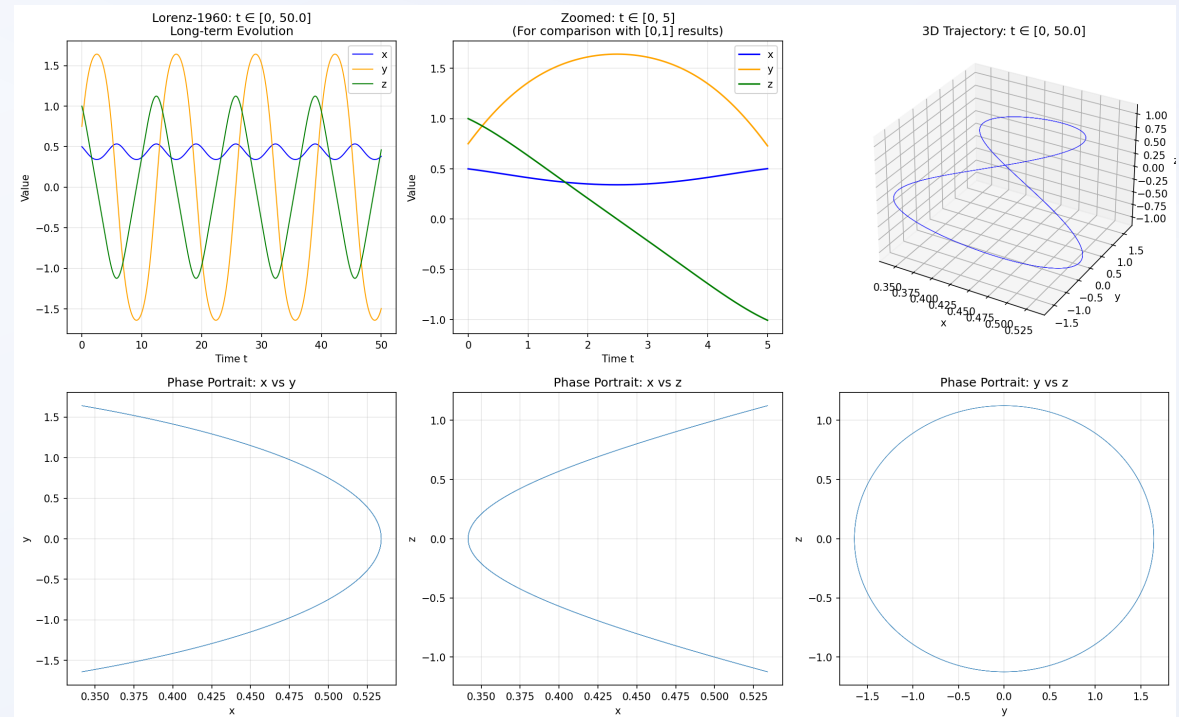
$$\dot{x} = -0.10 y z$$

$$\dot{y} = +1.60 x z$$

$$\dot{z} = -0.75 x y$$

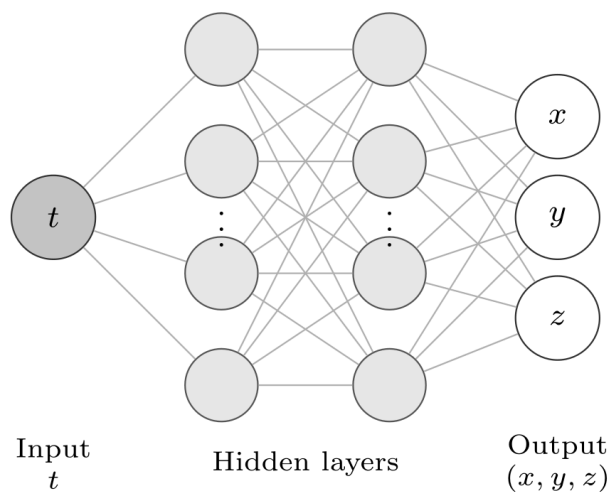
$k = 2, l = 1 \cdot x_0 = 0.5, y_0 = 0.75, z_0 = 1 \cdot t \in [0, 1]$

Three coupled nonlinear ODEs from an early meteorological model. Compact to write down, but hard to integrate accurately.

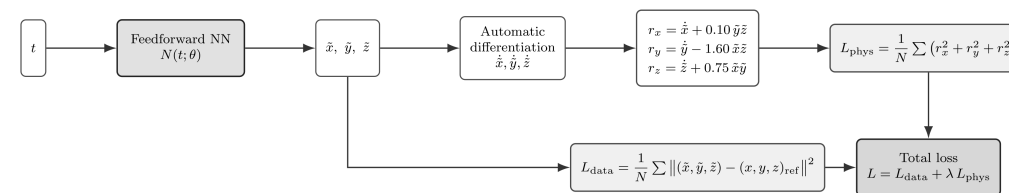


Long-term evolution over $t \in [0, 50]$: bounded, quasi-periodic orbits.

Plain ANN vs Physics-Informed NN



Plain ANN: learns $t \rightarrow (x, y, z)$ from data by backpropagation.



PINN: embeds the governing equations in the loss.

Our scope: the plain ANN on the left. PINNs appear only as a *literature benchmark*; we do not implement them.

Where the field stands

Work	Approach	Gap for our study
Matthews & Bihlo PinnDE	PINN + DeepONet on Lorenz-1960	Fixed architecture; relies on physics-informed loss
Aslam et al.	ANN + PSO on a Lorenz variant	Optimises weights, not architecture
Mall & Chakraverty	ANN, 4–6 nodes, sigmoid only	Simple ODEs; no depth / activation sweep
Lau & Werth ODEN	Plain ANN; tanh/sigmoid beat ReLU	No systematic depth–width study
Dubey / Audu et al.	Plain ANN on first-order ODEs	Fixed nets; not coupled, not Lorenz
Our work	Plain ANN, systematic search	Depth × width × activation on Lorenz-1960

The gap we fill



No architecture study

The Lorenz system has no systematic depth-width sweep for a plain ANN.



No PINN vs plain-ANN

Physics-informed and data-only nets are not compared on the same problem.



No activation study

Activation choice is rarely treated as a controlled variable on coupled ODEs.



Depth & width fixed

Prior work varies one factor at a time, never depth *and* width together.

What we ask

RQ1

Can a **plain ANN** learn the solution from numerical data alone, with no physics in the loss?

RQ2

Which **depth × width × activation** gives the best accuracy–cost trade-off?

RQ3

How close does the best plain ANN come to **physics-informed** baselines?

A two-phase comparative study

Phase 1 • Architecture search

depth 1-4

width 20 · 50 · 100

tanh

ReLU

sigmoid

GELU

Swish

60 runs · optimizer fixed (Adam)

Phase 2 • Optimizer comparison

3 best nets

Adam

L-BFGS

SGD + momentum

Architecture held constant; only the optimizer changes.

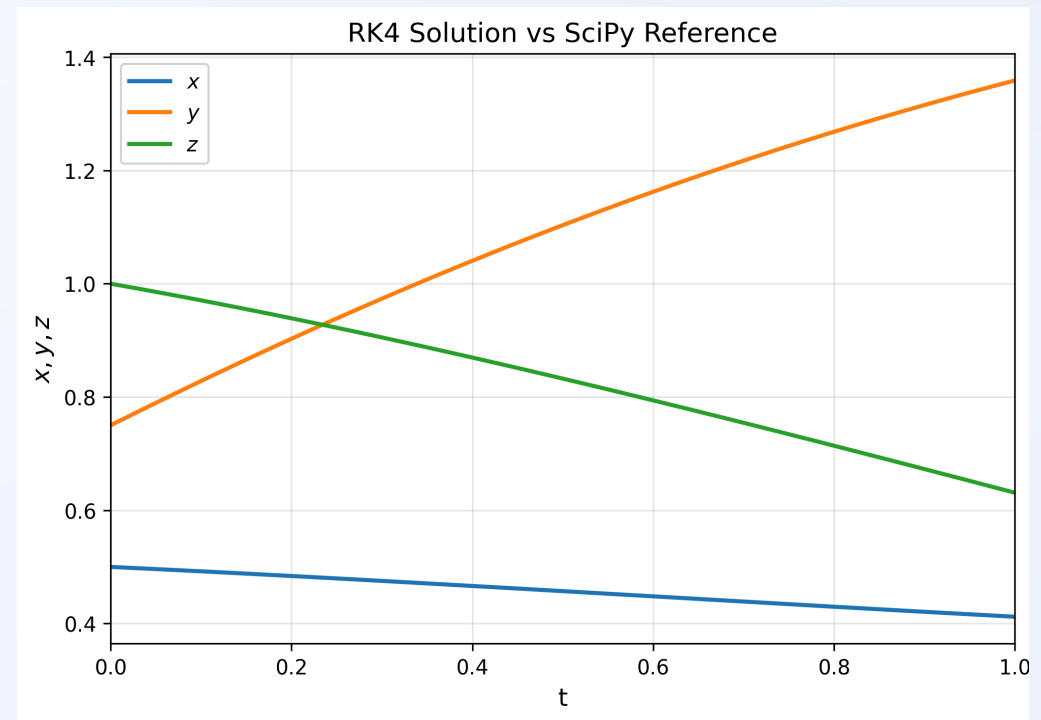
9 runs

69 total

Generate
RK4 + SciPyPreprocess
80/20 splitTrain
ANN gridEvaluate
5 metricsSelect
best

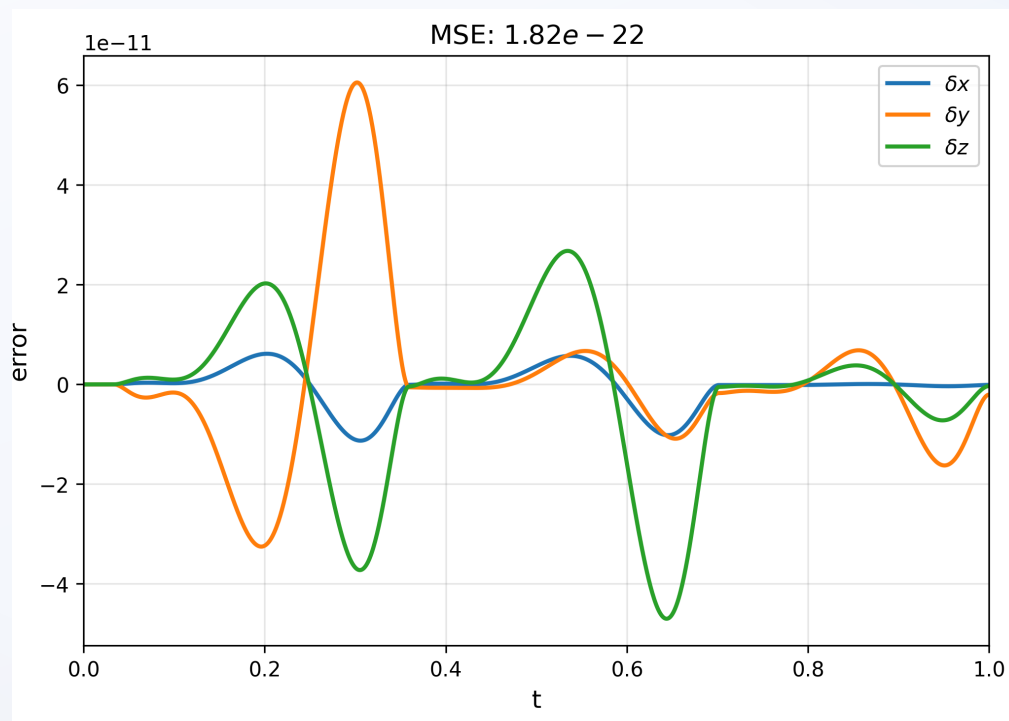
A baseline we can trust

- Custom **RK4** integrator: $h = 10^{-3}$, 1001 uniform points
- **SciPy DOP853**: $\text{rtol} = 10^{-10}$, $\text{atol} = 10^{-12}$
- 80 / 20 train-validation split, fixed seed 42
- Errors reported in original physical units



RK4 reference trajectory for x , y , z over $t \in [0, 1]$.

The baseline is verified



Pointwise RK4 - SciPy disagreement stays at the 10^{-11} level across the trajectory.

1.8×10^{-22}

solver-agreement MSE (RK4 vs SciPy)

✓ target was $< 10^{-10}$

The **ground truth** for every ANN we train in FYDP-II.

Roadmap & what's next



FYDP-I · WK 1-14

Done

Foundation

Problem, 17-paper review, requirements, and a verified dual-solver baseline.

FYDP-II · WK 15-28

Next

Experiments

Training pipeline, Phase-1 (60) then Phase-2 (9) runs, metrics & plots.

FYDP-III · WK 29-42

Planned

Analysis

Reproducibility, comparative analysis, final report & defense.

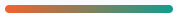
Immediate next steps

- Build the ANN training / evaluation pipeline
- Run the 60 Phase-1 architectures
- Pick the 3 best for Phase-2

Later extensions

- Repeated seeds for stability
- Denser grids & new initial conditions
- Direct comparison with PINN baselines

Where we stand



What's done

- A clear, narrow research question
- 17-paper review with an explicit gap
- Two-phase design (69 runs) locked
- Dual-solver baseline verified

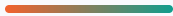
What it gives us

- First systematic plain-ANN architecture study on Lorenz-1960
- Clean attribution of architecture vs optimizer effects
- Narrow scope; accuracy claims wait for FYDP-II evidence

FYDP-II turns this validated baseline and locked design into measured results.

APPENDIX

Selected references



- 1 Lorenz, E.N. (1960). Maximum simplification of the dynamic equations. *Tellus* 12(3).
- 2 Matthews & Bihlo (2024). PinnDE: PINNs for ODEs and PDEs.
- 3 Raissi et al. (2019). Physics-informed neural networks. *JCP* 378.
- 4 Lagaris et al. (1998). ANN for solving ODEs/PDEs. *IEEE TNN* 9(5).
- 5 Lau & Werth (2020). ODEN: solving ODEs with neural networks.
- 6 Mall & Chakraverty (2013). ANN architectures for ODEs.
- 7 Dubey et al. (2024). Solving coupled ODEs with ANNs.
- 8 Chen et al. (2018). Neural ordinary differential equations. *NeurIPS*.

Full bibliography (17 works) in the project report · `paper/final/fydp.bib`



Team Paradox

Thank you

Questions & discussion

SUPERVISOR Dr. Muhammad Nomani Kabir · COURSE TEACHER Dr. Md. Motaharul Islam

Dept. of CSE · United International University